

Répartition des Tâches — Équipe de 5 Développeurs

Organisation de l'Équipe

#	Rôle	Nom	Module Principal
1	Chef de Projet / Coordinateur	À définir	Coordination, Intégration, Documentation
2	Data Engineer	À définir	Module 1 — Collecte & Base de données
3	IA Engineer #1	À définir	Module 2 — Analyse de Sentiment (Transformers)
4	IA Engineer #2	À définir	Module 2 — Extraction NLP (spaCy + Thèmes)
5	Développeur Frontend	À définir	Module 3 — Interface Web & Visualisations

DEV 1 — Chef de Projet / Coordinateur

Responsabilités

- Coordonner le travail de l'équipe et s'assurer que les modules s'intègrent correctement.
- Gérer le dépôt GitHub (branches, merges, code reviews).
- Rédiger le rapport technique (10-15 pages) et préparer la soutenance.
- Développer le **script de peuplement** (`seed_data.py`) et le **fichier principal de l'API** (`backend/main.py`) qui orchestre tous les modules.

Fichiers à Préparer

`backend/main.py` — Orchestration de l'API

C'est le fichier central qui **connecte** le travail des 4 autres développeurs. Il importe le moteur NLP (Dev 3 + Dev 4), utilise la base de données (Dev 2), et expose les endpoints consommés par le frontend (Dev 5).

Ce que ce fichier fait : - Initialise l'application FastAPI avec le middleware CORS. - Définit les **schémas Pydantic** (`ProductCreate`, `ReviewCreate`, `ReviewResponse`) pour la validation des données entrantes/sortantes. - Expose les routes REST de chaque module : - `POST /products/` et `GET /products/` → Gestion des produits (Module 1) - `POST /reviews/`, `POST /import/`, `POST /import-csv/` → Collecte des avis (Module 1) - `GET /reviews/` → Consultation avec filtres (Module 2) - `GET /stats/`, `GET /stats/timeline/`, `GET /stats/report/` → Rapport (Module 3) - Appelle `analyze_sentiment()` du moteur NLP à chaque avis ajouté.

Compétences mobilisées : FastAPI, Pydantic, intégration d'API, gestion de projet.

`seed_data.py` — Données de Démonstration

Ce script crée 3 produits/services et injecte 24 avis réalistes en français, avec des dates étalées sur 30 jours pour que les graphiques temporels soient visuellement riches lors de la démo.

Ce que ce fichier fait : - Se connecte à l'API en HTTP (vérifie que le backend est démarré). - Crée les produits via `POST /products/`. - Génère des dates aléatoires sur 30 jours. - Envoie les 24 avis via `POST /import/` pour analyse automatique.

`requirements.txt` — Dépendances

Liste les bibliothèques Python nécessaires au projet.

Rapport Technique (10-15 pages)

Structure recommandée : 1. Introduction / Contexte 2. Problématique 3. Architecture technique (schéma + justification des choix) 4. Présentation des 3 modules 5. Démonstration (captures d'écran annotées) 6. Difficultés rencontrées et solutions 7. Limites et améliorations futures 8. Conclusion

DEV 2 — Data Engineer

Responsabilités

- Concevoir et implémenter le **schéma de la base de données**.
- Gérer la **collecte, le nettoyage et la structuration** des données textuelles.
- Développer la logique d'**importation CSV** (parsing, validation, normalisation).
- Préparer des **jeux de données de test** réalistes.

Fichiers à Préparer

`backend/database.py` — Schéma de la Base de Données

Ce que ce fichier fait : - Configure la connexion SQLite via SQLAlchemy. - Définit 2 modèles (tables) : - `Product` : id, name, description, created_at — représente un produit ou service de l'agence. - `Review` : id, source, text, sentiment, score, confidence, keywords, themes, product_id (FK), date — un avis client analysé. - La relation `Product` ↔ `Review` est un **One-to-Many** (un produit a plusieurs avis). - Exporte les fonctions `init_db()` et `get_db()` utilisées par l'API.

Concepts clés à maîtriser : - **ORM (Object-Relational Mapping)** — SQLAlchemy traduit les classes Python en tables SQL. - **Foreign Key** — `product_id` dans `Review` pointe vers `Product.id`. - **Session management** — `get_db()` est un générateur Python qui ouvre et ferme proprement la session.

Logique CSV dans `main.py` (endpoint `/import-csv/`)

Le Dev 2 doit comprendre la logique de parsing CSV dans le endpoint `import_csv` : - Réception du fichier via `UploadFile`. - Décodage UTF-8 (avec BOM via `utf-8-sig`). - Lecture avec `csv.DictReader` — chaque ligne est un dictionnaire. - Extraction de la colonne obligatoire `text`, et des colonnes optionnelles `source`, `date`, `product_id`. - Appel au moteur NLP pour chaque ligne, puis insertion en base.

Données de test

Préparer au moins 2 fichiers CSV de test : - `data/avis_google.csv` : Avis simulés provenant de Google Reviews. - `data/avis_whatsapp.csv` : Messages WhatsApp simulés.

Format attendu :

```
text,source,product_id "Le service est excellent, je recommande",Google,1 "Très déçu de la qualité",Google,1
```

Compétences mobilisées : Data engineering, SQL, CSV parsing, nettoyage de données.

DEV 3 — IA Engineer #1 (Analyse de Sentiment)

Responsabilités

- Implémenter le **pipeline d'analyse de sentiment** avec Hugging Face Transformers.
- Choisir, charger et configurer le modèle pré-entraîné.
- Convertir les scores bruts du modèle en polarité exploitable (Positif/Négatif/Neutre).
- Mesurer et optimiser la **précision** de la classification.

Fichier à Préparer

[backend/nlp_engine.py](#) — Partie Sentiment (lignes 1-148)

Ce que ce code fait, étape par étape :

Chargement du modèle (au démarrage du serveur) : `python sentiment_analyzer = pipeline("sentiment-analysis", model="nlpTown/bert-base-multilingual-uncased-sentiment", )` La fonction `pipeline()` de Hugging Face télécharge le modèle BERT depuis le Hub (environ 680 Mo) et le charge en mémoire. Cela prend ~5-10 secondes au premier lancement.

Analyse d'un texte : `python res = sentiment_analyzer(text[:512])[0] # res = {"label": "4 stars", "score": 0.876}`

3. Le texte est tronqué à 512 tokens (limite de BERT).

Le modèle retourne un label (1-5 stars) et un score de confiance (0-1).

Conversion en polarité : `python if score_val >= 4: sentiment = "Positif" elif score_val <= 2: sentiment = "Négatif" else: sentiment = "Neutre"`

Concepts clés à maîtriser : - **Transformers** — Architecture de réseau de neurones basée sur le mécanisme d'attention. - **BERT** — Modèle pré-entraîné de Google, fine-tuné ici sur des avis produits. - **Pipeline** — Abstraction de Hugging Face qui encapsule tokenizer + modèle + post-traitement. - **Truncation** — Les modèles BERT ont une limite de 512 tokens, les textes plus longs doivent être tronqués.

Compétences mobilisées : NLP, Transformers, analyse de sentiment, classification de texte.

DEV 4 — IA Engineer #2 (Extraction NLP + Thèmes)

Responsabilités

- Implémenter l'**extraction de mots-clés** avec spaCy.
- Développer le système de **détection de thèmes récurrents**.
- Créer les fonctions d'**agrégation** pour le rapport (points forts / points d'amélioration).

Fichier à Préparer

backend/nlp_engine.py — Partie Extraction (lignes 40-185)

Fonction `extract_keywords(text)` :

1. spaCy analyse le texte et produit un objet `Doc` avec toutes les annotations linguistiques.
2. On itère sur les **Noun Chunks** (groupes nominaux) :
3. "Le service client est lent" → Noun chunk = "Le service client"
4. On retire les déterminants (DET) → "service client"
5. On filtre les pronoms et les chunks trop courts.
6. On ajoute les **adjectifs isolés** de plus de 3 lettres ("professionnel", "lent", "rapide").
7. On déduplique et on retourne les 8 premiers.

Fonction `detect_themes(text)` :

Correspondance lexicale avec un dictionnaire prédéfini de 8 thèmes :

```
THEME_KEYWORDS = { "Service Client": ["service", "support", "aide", "réponse", ...], "Prix & Tarifs": ["prix", "tarif", "cher", "coût", ...], # ... 6 autres thèmes }
```

Pour chaque thème, si au moins un mot de la liste est présent dans le texte, le thème est détecté.

Fonction `aggregate_themes(reviews_data)` :

Compte les occurrences de chaque thème sur l'ensemble des avis. Retourne un classement trié pour le rapport.

Fonction `extract_strengths_weaknesses(reviews_data)` :

Sépare les mots-clés en 2 groupes : - **Points forts** : mots-clés des avis avec `sentiment == "Positif"` - **Points d'amélioration** : mots-clés des avis avec `sentiment == "Négatif"`

Retourne les 8 termes les plus fréquents de chaque catégorie.

Compétences mobilisées : spaCy, POS tagging, dependency parsing, agrégation de données.

DEV 5 — Développeur Frontend

Responsabilités

- Développer l'**interface web React** complète (3 pages).
- Implémenter le **design system CSS** (Dark Mode, Glassmorphism).
- Créer les **graphiques interactifs** avec Recharts.
- Intégrer l'**API backend** via Axios.

Fichiers à Préparer

frontend-premium/src/index.css — Design System

Ce fichier contient **tout le CSS du projet** : - Variables CSS (couleurs, border-radius, ombres). - Classes de layout (`.app-layout`, `.sidebar`, `.main-content`). - Classes Glassmorphism (`.card` avec `backdrop-filter: blur()`). - Grilles responsives (`.kpi-row`, `.charts-row`, `.reviews-grid`). - Styles de badges sentiment (`.badge-positif`, `.badge-negatif`, `.badge-neutre`). - Animations (`.fade-in`, `.spin`). - Styles de formulaires et modals.

frontend-premium/src/App.jsx — Application Principale

Contient 3 pages internes : - **DashboardPage** — KPIs, graphiques sentiments/sources, timeline, thèmes, avis récents. - **ReviewsPage** — Grille complète de tous les avis avec filtres. - **ReportPage** — Rapport structuré en 4 sections numérotées.

La navigation entre pages se fait via un state React (page), pas de router.

Composants (src/components/)

Composant	Rôle	Complexité
KpiCard.jsx	Carte indicateur (Total, Score, Taux, Sources)	Simple
SentimentChart.jsx	Donut chart + Bar chart (Recharts)	Moyenne
TimelineChart.jsx	Area chart évolution temporelle	Moyenne
ReviewCard.jsx	Carte d'avis (badge, étoiles, tags)	Moyenne
ReportPanel.jsx	Panneau avec barres de progression	Simple
AddReviewModal.jsx	Modal formulaire + affichage résultat IA	Complexe
ImportCsvModal.jsx	Modal drag & drop CSV	Complexe

Compétences mobilisées : React, CSS avancé, Recharts, Axios, UX/UI design.

Planning de Coordination

Semaine	Dev 1 (Coord)	Dev 2 (Data)	Dev 3 (IA Sent.)	Dev 4 (IA NLP)	Dev 5 (Front)
S1	Analyse du besoin, setup Git	Conception BDD	Recherche modèles	Recherche spaCy	Maquettes UI
S2	Schémas Pydantic	database.py + CSV test	Téléchargement modèle	Installation spaCy	Setup Vite + CSS
S3	Intégration main.py	Endpoint CSV	analyze_sentiment(extract_keywords())	Composants KPI	
S4	Tests d'intégration	Nettoyage données	Optimisation scores	detect_themes()	Composants Charts
S5	seed_data.py	Tests import CSV	Tests précision	Agrégation rapport	Pages Dashboard
S6	Endpoints rapport	Jeux de données finaux	Documentation IA	Documentation NLP	Modals + Review
S7	Tests complets	Corrections bugs	Corrections bugs	Corrections bugs	Polish UI

S8	Rapport + Soutenance	Relecture rapport	Relecture rapport	Relecture rapport	Démo finale
----	-------------------------	----------------------	----------------------	----------------------	-------------

Règles de Coordination

1. **Branche Git par développeur** : `dev/nom-prenom` → merge dans `main` après review.
2. **Communication** : Le Dev 2, Dev 3 et Dev 4 doivent se synchroniser car ils travaillent tous sur le backend.
3. **Interface commune** : La fonction `analyze_sentiment(text) → dict` est le **contrat** entre le backend et le frontend. Sa signature ne doit pas changer.
4. **Tests** : Chaque développeur teste son module indépendamment avant l'intégration.
5. **Documentation** : Chaque fichier doit contenir un docstring explicatif en en-tête.